

Nom:

Prénom:

Note:

Dans tout le devoir, sauf indication contraire, on considèrera que les nombres sont des entiers relatifs codés sur 8 bits exactement. On supposera que les variables d'entrée de chaque algorithme sont conformes aux attentes de l'algorithme.

Exercice 1 ►

/4 points

1. Sans aucune justification, donner le nombre d'états possibles codables avec:
 - a) 3 bits $2^3 = 8$
 - b) 8 bits $2^8 = 256$
 - c) n bits 2^n
2. On code maintenant un nombre entier relatif avec la méthode du complément à 2 vue en classe. Quelle est la valeur minimale et maximale décimale codable avec:
 - a) 3 bits entre $-2^2 = -4$ et $2^2 - 1 = 3$
 - b) 8 bits entre $-2^7 = -128$ et $2^7 - 1 = 127$
 - c) n bits entre -2^{n-1} et $2^{n-1} - 1$
3. Que peut-on dire de l'entier relatif codé sur 8 bits en complément à deux suivant: 1001 0101. Sa valeur décimale est-elle: (plusieurs réponses possibles)

☐ paire☒ impaire☐ positive☒ négative

Exercice 2 ►

/2 points

Exercice 3 ►

/2 points

Écrire l'algorithme de conversion d'un entier naturel compris entre 0 et 255 en binaire sur 8 bits en pseudo code ou en Python.

```
def conversion_dec_to_bin(val_dec):
    binaire = ''
    for i in range(8):
        reste = val_dec % 2
        binaire = str(reste) + binaire
        val_dec = val_dec // 2
    return binaire
```

Citez les différentes étapes de la conversion d'un entier relatif en binaire sur 8 bits. Vous choisirez de développer un des deux algorithmes vus en cours. Inutile de l'écrire en Python.

- Si l'entier est positif, on le convertit en binaire sur 8 bits avec la méthode de la question 1.
- Si l'entier est négatif,

Méthode 1:

- On prend la valeur absolue de l'entier.
- On le convertit en binaire sur 8 bits avec la méthode de la question 1.
- On inverse tous les bits du résultat.
- On additionne 1 au résultat précédent.
- On obtient le complément à 2 de l'entier.

Méthode 2:

- On prend la valeur absolue de l'entier.
- On le convertit en binaire sur 8 bits avec la méthode de la question 1.
- On recopie les bits de la droite vers la gauche jusqu'au premier '1' inclus
- On inverse tous les bits à gauche du premier '1' inclus.
- On obtient le complément à 2 de l'entier.

Exercice 4 ►

/2 points

Convertir le nombre entier relatif suivant avec la méthode du complément à 2 sur 8 bits. Justifier.

-56

$$|-56| = 56 = 0011\ 1000_2$$

Méthode 1:

- Inversion des bits: 1100 0111₂
- Ajouter 1: 1100 1000₂

Méthode 2:

- Recopie des bits jusqu'au 1er '1' inclus
- Inversion des bits restants 1100 1000₂

Exercice 5 ►

/1 point

Convertir le nombre entier relatif suivant avec la méthode du complément à 2 sur 8 bits. Justifier.

112

112 est un nombre positif donc il se convertit comme les entiers naturels.

$$112 = 64 + 32 + 16 = 0111\ 0000_2$$

Exercice 6 ►

/4 points

On suppose que l'on dispose des fonctions suivantes en Python (il est inutile d'écrire le code de ces fonctions). Ecrire la fonction `conversion_relatif_binaire` qui convertit un entier relatif en binaire sur 8 bits. Vous utiliserez les fonctions précédentes pour créer votre fonction. Vous appliquerez la méthode de votre choix (la méthode 1 est plus simple !)

```
def conversion_relatif_binaire(val_entiere):                # METHODE 1
    if val_entiere >= 0:
        return conversion_entier_binaire(val_entiere)
    else:
        val_entiere = - val_entiere
        binaire = conversion_entier_binaire(val_entiere)

        binaire_inverse = ''
        for bit in binaire:
            binaire_inverse = binaire_inverse + inverse_bit(bit)
        return additionne_binaire(binaire_inverse)

def conversion_relatif_binaire(val_entiere):                # METHODE 2
    if val_entiere >= 0:
        return conversion_entier_binaire(val_entiere)
    else:
        val_entiere = - val_entiere
        binaire = conversion_entier_binaire(val_entiere)

        resultat = ''
        i = len(binaire) - 1
        premier_1_trouve = False
        # On parcourt le binaire de droite à gauche
        while i >= 0:
            bit = binaire[i]

            if not premier_1_trouve:
                resultat = bit + resultat
                if bit == '1':
                    premier_1_trouve = True
            else:
                resultat = inverse_bit(bit) + resultat
            i = i - 1
```